Functions

April 14, 2026

1 Functions

1.1 Contents

- Classes of Functions
  - Continuous Functions
  - Lp Spaces
- Polynomials, Power Series, and Rational Functions
  - Polynomials
  - Degree of a Polynomial
  - Set of All Polynomials
  - Polynomial Divisibility and Modular Arithmetic
  - Special Polynomial Families
  - Power Series
  - Rational Functions
- Miscellany
  - Arg Min and Arg Max
  - One-to-One, Onto, and Bijection

1.2 Notes

- The notebooks are largely self-contained. If a symbol appears, an explanation will be present at some point in the same notebook — most often with a link to the cell where it is explained. Only if a symbol cannot be covered in this notebook will a reference to another notebook be provided.
- **GitHub does a poor job of rendering these notebooks.** The online render is missing links, symbols are absent, and notations are badly formatted. Clone a local copy or download the notebook and open it locally, or refer to the PDFs in the `PDFs/` folder.
- All code uses only Python's standard library.
- **See the Collections notebook before this notebook to gain familiarity with set notations.**

1.3 Importing Libraries

[1]:
```
import math
```

Fundamentals

Set Map

If A and B are any sets, the notation $f : A \to B$ means that f is a function that takes as input values from the set A and returns values in the set B. The set A is called the *domain* and B is called the *codomain*.

$$f : A \to B$$

It is not necessary that every value of B be realized as an output. For example, $\cos : \mathbb{R} \to \mathbb{R}$ takes any real number as input and returns a real number, but the outputs lie only in the interval $[-1, 1]$. (See: Image of a Function)

In many physical sciences and engineering, most functions map real numbers to real numbers: $f : \mathbb{R} \to \mathbb{R}$.

[2]:
```python
def inverse(a):
    return 1 / a

A = {2, 4, 8}
B = {0.125, 0.25, 0.5, 1, 4, 15}

for a in A:
    print(f'inverse({a}) = {inverse(a)}, in B: {inverse(a) in B}')
```

```
inverse(8) = 0.125, in B: True
inverse(2) = 0.5, in B: True
inverse(4) = 0.25, in B: True
```

Note 1: Decorations may appear above the arrow to indicate properties of the function such as one-to-one: $f : A \overset{1:1}{\to} B$, or the arrow itself may be decorated: $f : A \hookrightarrow B$ (injection) or $f : A \twoheadrightarrow B$ (surjection). (See: One-to-One, Onto, and Bijection)

Note 2: Shorthand for multiple functions sharing the same domain and codomain uses a comma: $f, g : \mathbb{R} \to \mathbb{R}$.

Image of a Function

For a function $f : X \to Y$, the *image* of f is the subset of Y consisting of all values actually produced by f. It is denoted $\mathrm{Im}(f)$.

$$\mathrm{Im}(f) = \{y \in Y \mid \exists\, x \in X,\ f(x) = y\}$$

More generally, if $A \subseteq X$, then $f(A)$ stands for the set $\{f(a) \mid a \in A\}$, so $f(X) = \mathrm{Im}(f)$.

For more information on the there-exists $\exists$ notation, see the Logic notebook.

[3]:
```python
def square(x):
    return x ** 2

X = {-3, -2, -1, 0, 1, 2, 3}
image = {square(x) for x in X}

print('X:', sorted(X))
print('Im(f):', sorted(image))
```

```
X: [-3, -2, -1, 0, 1, 2, 3]
Im(f): [0, 1, 4, 9]
```

Kernel of a Function

For a linear mapping $f : A \to B$, the *kernel* of f is the set of all elements in A that map to zero. It is denoted $\ker f$.

$$\ker f = \{\mathbf{x} \in A \mid f(\mathbf{x}) = 0\}$$

The kernel of a matrix is also called the *null space*.

For more information on the kernel of a matrix, see the Linear Algebra notebook.

[4]:
```
def f(x):
    return x ** 2 - 4

X = [-5, -2, -1, 0, 1, 2, 5]
kernel = [x for x in X if f(x) == 0]

print('ker f:', kernel)
```

```
ker f: [-2, 2]
```

Value Notation

The notation $f(x)$ indicates the value returned by the function f when evaluated at x. That is, f is the function while $f(x)$ is the value (typically a number) returned by the function.

$$y = f(x)$$

Functions that accept multiple arguments are denoted $y = f(x_1, x_2, \ldots, x_n)$. In common usage, a *parameter* is a quantity that influences the output but is viewed as held constant. For example, a function with argument x and parameter a:

$$y = f(x, a) = a \sin(x)$$

[5]:
```
f = math.sin

x = 0.5
y = f(x)
y, x
```

[5]:
```
(0.479425538604203, 0.5)
```

[6]:
```
def f(x, a):
    return a * math.sin(x)

x, a = 0.5, 3
y = f(x, a)
y, x, a
```

[6]:
```
(1.438276615812609, 0.5, 3)
```

Note: An alternative notation for function evaluation uses the *evaluation bar*. The general form is:

$$\text{expression involving } x \,\Big|_{x=a}$$

This means to evaluate the expression by substituting the value a for the variable x. It is used especially with Leibniz notation for derivatives: $\frac{df}{dx}\Big|_{x=a}$.

Dot Notation

The notation $f(\cdot)$ is used to emphasize that f is a function with the dot showing that an argument is expected. This is handy when referring to a function specified by non-letter symbols. For example, the absolute value function may be written $|\cdot|$ rather than putting in a dummy variable.

$$f(\cdot, b)$$

represents a function whose first argument varies over its domain while the second argument is held constant at b.

[7]:
```
def f(x, b=5):
    return b * x ** 2

X = [1, 2, 3, 4]

# f(., b) with b held constant at 5
print('X:', X)
print('f(X, 5):', [f(x) for x in X])
```

```
X: [1, 2, 3, 4]
f(X, 5): [5, 20, 45, 80]
```

Map Notation

The notation $y = f(x)$ is sometimes abbreviated to $x \mapsto y$, pronounced "x maps to y." The function f is absent from this expression, so it is used only when the function under consideration is unambiguously known.

$$x \mapsto y$$

Alternatively, $x \overset{f}{\mapsto} y$ makes the function explicit.

Note: The expression $x \mapsto y$ differs from $x \to y$ in that the latter denotes the map between sets ($f : A \to B$) while the former denotes the map between elements.

[8]:
```
f = math.cos
```

```
x = math.pi / 3
y = f(x)

print(f'x = {x:.4f} maps to y = {y:.4f}')
```

```
x = 1.0472 maps to y = 0.5000
```

Operator / Transform

A function whose inputs and outputs are themselves functions is often called an *operator* or a *transform*. To distinguish operators from functions of numbers, the operator name is typically capitalized and the parentheses are omitted. The application of operator L to function f is denoted:

$$Lf$$

A specific example is the Laplace transform operator $\mathcal{L}$, which maps a temporal function $f(t)$ to a function in the complex domain $F(s)$, so that $\mathcal{L}f = F$.

Piecewise Notation

A function may be defined piecewise when no single algebraic formula covers the entire domain. Each piece specifies the formula and the subdomain over which it applies.

$$f(x) = \begin{cases} e^{-1/x^2} & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

[9]:
```
def f(x):
    if x > 0:
        return math.exp(-1 / x ** 2)
    return 0

for x in [-2, -1, 0, 0.5, 1, 2]:
    print(f'f({x:4}) = {f(x):.6f}')
```

```
f(  -2) = 0.000000
f(  -1) = 0.000000
f(   0) = 0.000000
f( 0.5) = 0.018316
f(   1) = 0.367879
f(   2) = 0.778801
```

Composition of Functions

If $f : A \to B$ and $g : B \to C$, then the *composition* of g and f is a new function $(g \circ f) : A \to C$ defined by:

$$(g \circ f)(x) = g(f(x))$$

The order matters: $(f \circ g)(x) = f(g(x))$ is generally not the same as $(g \circ f)(x)$.

[10]:
```
def f(x):
    return 2 * x + 5

def g(x):
    return x ** 2

a = 3
print(f'(g o f)({a}) = g(f({a})) = g({f(a)}) = {g(f(a))}')
print(f'(f o g)({a}) = f(g({a})) = f({g(a)}) = {f(g(a))}')
```

```
(g o f)(3) = g(f(3)) = g(11) = 121
(f o g)(3) = f(g(3)) = f(9) = 23
```

Inversion of Functions

The notation $f^{-1}(y)$ stands for the value x such that $f(x) = y$; the function f^{-1} is called the *inverse* of f.

$$f^{-1}(y) = x \quad \text{where } f(x) = y$$

If more than one x maps to the same y, then f^{-1} is not a function in general. Some authors coerce it into a function by restricting the domain. For example, arcsin is $\sin^{-1}$ restricted to $x \in [-\frac{\pi}{2}, \frac{\pi}{2}]$.

Warning: $f^{-1}(y)$ does **not** mean $\frac{1}{f(y)}$. (See: Function Exponentiation)

The notation f^{-1} may also be applied to a set. If $B \subseteq Y$ and $f : X \to Y$, then:

$$f^{-1}(B) = \{x \in X \mid f(x) \in B\}$$

[11]:
```
def f(x):
    return math.exp(x)

def f_inv(y):
    return math.log(y)

x = 2.0
y = f(x)
print(f'f({x}) = {y:.4f}')
print(f'f_inv({y:.4f}) = {f_inv(y):.4f}')
```

```
f(2.0) = 7.3891
f_inv(7.3891) = 2.0000
```

Function Exponentiation

The notation $f^n(x)$ generally means the n-th power of the function's output:

$$f^2(x) = \big[f(x)\big]^2$$

This is used especially with trigonometric functions: $\sin^2\theta = (\sin\theta)^2$.

Warning: This notation creates an inconsistency with f^{-1}, which denotes the *inverse* rather than $\big[f(x)\big]^{-1}$. In some contexts, $f^2(x)$ may instead mean $(f \circ f)(x) = f(f(x))$, which *is* consistent with the inverse notation. Context determines the intended meaning. (See: Composition of Functions and Inversion of Functions)

[12]:
```
x = math.pi / 4
sin_squared = math.sin(x) ** 2
cos_squared = math.cos(x) ** 2

print(f'sin^2({x:.4f}) = {sin_squared:.4f}')
print(f'cos^2({x:.4f}) = {cos_squared:.4f}')
print(f'sin^2 + cos^2 = {sin_squared + cos_squared:.4f}')
```

```
sin^2(0.7854) = 0.5000
cos^2(0.7854) = 0.5000
sin^2 + cos^2 = 1.0000
```

Standard Functions

Absolute Value

For a real or complex number x, $|x|$ is the absolute value of x — its non-negative magnitude.

$$|x| = \begin{cases} x & \text{if } x \geq 0 \\ -x & \text{if } x < 0 \end{cases}$$

For more information on absolute value, see the Numbers notebook.

[13]:
```
abs(7), abs(-7), abs(0)
```

[13]:
```
(7, 7, 0)
```

Square and Other Roots

For nonnegative real numbers x, the notation $\sqrt{x}$ is unambiguously the nonnegative square root of x. More generally, for $x \geq 0$ and $n > 0$, $\sqrt[n]{x}$ is the nonnegative n^{th}-root of x.

These may be written in exponential form:

$$\sqrt{x} = x^{1/2} \qquad \sqrt[n]{x} = x^{1/n}$$

When x is a negative real number, $\sqrt{x} = i\sqrt{|x|}$. If n is an odd integer, $\sqrt[n]{x}$ is the unique real number y so that $y^n = x$.

[14]:
```
print('sqrt(16)  =', math.sqrt(16))
print('cbrt(27)  =', round(27 ** (1/3), 4))
print('4th root of 81 =', round(81 ** (1/4), 4))
```

```
sqrt(16)  = 4.0
cbrt(27)  = 3.0
4th root of 81 = 3.0
```

Signum

The notation $\operatorname{sgn} x$ is the *sign* of x, also called the *signum* function.

$$\operatorname{sgn} x = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x = 0 \\ -1 & \text{if } x < 0 \end{cases}$$

[15]:
```
def sgn(x):
    if x > 0: return 1
    if x == 0: return 0
    return -1

for x in [-5, -0.1, 0, 0.1, 5]:
    print(f'sgn({x:5}) = {sgn(x):2}')
```

```
sgn(   -5) = -1
sgn( -0.1) = -1
sgn(    0) =  0
sgn(  0.1) =  1
sgn(    5) =  1
```

Exponential Function

The notation $\exp x$ stands for e^x, where $e \approx 2.71828$ is Euler's number. The parenthesized form $\exp(x)$ is also common; when the argument is a more complex expression, parentheses are preferred: $\exp(x^2 + y^2)$.

[16]:
```
print('exp(0) =', math.exp(0))
print('exp(1) =', math.exp(1))
print('exp(2) =', math.exp(2))
print('e      =', math.e)
```

```
exp(0) = 1.0
exp(1) = 2.718281828459045
exp(2) = 7.38905609893065
e      = 2.718281828459045
```

Logarithms

The notation $\log x$ is somewhat ambiguous. In science and engineering it typically means the base-10 logarithm; in mathematics it typically means the base-e (natural) logarithm.

The unambiguous notations are:

- $\ln x$ — natural logarithm (base e): $\ln x = \log_e x$
- $\log_{10} x$ — common logarithm (base 10)
- $\log_2 x$ or $\lg x$ — binary logarithm (base 2)

The function $\log^* x$ occurs in computer science. It stands for the number of times one must apply $\log_2$ to x to reach a result that is less than or equal to 1. It is called the *log star* function.

[17]:
```
x = 100
print(f'ln({x})     = {math.log(x):.4f}')
print(f'log10({x})  = {math.log10(x):.4f}')
print(f'log2({x})   = {math.log2(x):.4f}')
```

```
ln(100)     = 4.6052
log10(100)  = 2.0000
log2(100)   = 6.6439
```

[18]:
```
# log star: number of times log2 must be applied to reach <= 1
def log_star(x):
    count = 0
    while x > 1:
        x = math.log2(x)
        count += 1
    return count

for n in [2, 4, 16, 65536]:
    print(f'log*({n}) = {log_star(n)}')
```

```
log*(2) = 1
log*(4) = 2
log*(16) = 3
log*(65536) = 4
```

Trigonometric Functions

The standard trigonometric functions are sin, cos, tan, sec, csc, and cot. In mathematics the arguments are expressed in radians; in engineering they may be expressed in degrees.

The notation $\sin^2 x$ means $(\sin x)^2$. However, $\sin^{-1} x$ does **not** mean $1/\sin x$. Rather, $\sin^{-1} x$ is the *inverse sine* (or *arc sine*) function. It may also be written $\arcsin x$. Likewise for arccos, arctan, and so forth.

$$\sin^2 x + \cos^2 x = 1$$

[19]:
```
x = math.pi / 6

print(f'sin(pi/6) = {math.sin(x):.4f}')
print(f'cos(pi/6) = {math.cos(x):.4f}')
print(f'tan(pi/6) = {math.tan(x):.4f}')
print(f'sin^2 + cos^2 = {math.sin(x)**2 + math.cos(x)**2:.4f}')
```

```
sin(pi/6) = 0.5000
cos(pi/6) = 0.8660
tan(pi/6) = 0.5774
sin^2 + cos^2 = 1.0000
```

[20]:
```
# inverse trigonometric functions
y = 0.5
print(f'arcsin({y}) = {math.asin(y):.4f} rad')
print(f'arccos({y}) = {math.acos(y):.4f} rad')
print(f'arctan({y}) = {math.atan(y):.4f} rad')
```

```
arcsin(0.5) = 0.5236 rad
arccos(0.5) = 1.0472 rad
arctan(0.5) = 0.4636 rad
```

Hyperbolic Trigonometric Functions

The standard trigonometric functions have hyperbolic counterparts, denoted by appending 'h' to their names: sinh, cosh, tanh, sech, csch, coth.

$$\sinh x = \frac{e^x - e^{-x}}{2} \qquad \cosh x = \frac{e^x + e^{-x}}{2}$$

Also: $\tanh x = \sinh x / \cosh x$, $\operatorname{sech} x = 1/\cosh x$, $\operatorname{csch} x = 1/\sinh x$, and $\coth x = \cosh x / \sinh x$.

As with ordinary trig, $\sinh^2 x$ means $(\sinh x)^2$, while $\sinh^{-1} x$ is the inverse function of sinh.

[21]:
```
x = 1.0
print(f'sinh({x}) = {math.sinh(x):.4f}')
print(f'cosh({x}) = {math.cosh(x):.4f}')
print(f'tanh({x}) = {math.tanh(x):.4f}')

# verify definition
print(f'(e^x - e^-x)/2 = {(math.exp(x) - math.exp(-x)) / 2:.4f}')
```

```
sinh(1.0) = 1.1752
cosh(1.0) = 1.5431
tanh(1.0) = 0.7616
(e^x - e^-x)/2 = 1.1752
```

Sine Cardinal

The function $\operatorname{sinc} x$ is called the *sine cardinal*. It is variously defined as:

$$\operatorname{sinc} x = \frac{\sin x}{x} \qquad \text{or} \qquad \operatorname{sinc} x = \frac{\sin \pi x}{\pi x}$$

for $x \neq 0$, with $\operatorname{sinc} 0 = 1$.

[22]:
```
def sinc(x):
    if x == 0:
        return 1
    return math.sin(x) / x

for x in [0, 0.5, 1.0, math.pi, 2 * math.pi]:
    print(f'sinc({x:6.3f}) = {sinc(x): .6f}')
```

```
sinc( 0.000) =  1.000000
sinc( 0.500) =  0.958851
sinc( 1.000) =  0.841471
sinc( 3.142) =  0.000000
sinc( 6.283) = -0.000000
```

Floor and Ceiling

The notation $\lfloor x \rfloor$ stands for the *floor* of x: the result of rounding x down to the nearest integer. The notation $\lceil x \rceil$ stands for the *ceiling* of x: the result of rounding x up to the nearest integer.

$$\lfloor e \rfloor = \lfloor 3 \rfloor = \lfloor \pi \rfloor = 3 \qquad \lceil e \rceil = \lceil \pi \rceil = 3$$

Note: In older books, the floor is written $[x]$ and the ceiling is written $\{x\}$.

[23]:
```
for x in [2.3, 2.7, -1.5, math.pi, math.e]:
    print(f'floor({x:6.3f}) = {math.floor(x):3},  ceil({x:6.3f}) = {math.
 ↪ceil(x):2}')
```

```
floor( 2.300) =   2,  ceil( 2.300) =  3
floor( 2.700) =   2,  ceil( 2.700) =  3
floor(-1.500) =  -2,  ceil(-1.500) = -1
floor( 3.142) =   3,  ceil( 3.142) =  4
floor( 2.718) =   2,  ceil( 2.718) =  3
```

Error Function

The error function $\operatorname{erf} x$ is a function that occurs in probability theory. It is defined by:

$$\operatorname{erf} x = \frac{2}{\sqrt{\pi}} \int_0^x \exp(-t^2)\, dt$$

[24]:
```
for x in [0, 0.5, 1.0, 2.0, 3.0]:
    print(f'erf({x:.1f}) = {math.erf(x): .6f}')
```

```
erf(0.0) =  0.000000
erf(0.5) =  0.520500
erf(1.0) =  0.842701
erf(2.0) =  0.995322
erf(3.0) =  0.999978
```

Gamma Function

The Gamma function $\Gamma(x)$ is defined by:

$$\Gamma(x) = \int_0^\infty t^{\,x-1}\, e^{-t}\, dt$$

It is closely related to the factorial function: for any positive integer n, $\Gamma(n) = (n-1)!$. More generally, $\Gamma(x+1) = x\,\Gamma(x)$ for all $x > 0$.

[25]:
```
for n in range(1, 8):
    print(f'Gamma({n}) = {math.gamma(n):.1f},  (n-1)! = {math.factorial(n -␣
 ↪1)}')
```

```
Gamma(1) = 1.0,  (n-1)! = 1
Gamma(2) = 1.0,  (n-1)! = 1
Gamma(3) = 2.0,  (n-1)! = 2
Gamma(4) = 6.0,  (n-1)! = 6
Gamma(5) = 24.0,  (n-1)! = 24
Gamma(6) = 120.0,  (n-1)! = 120
Gamma(7) = 720.0,  (n-1)! = 720
```

(See: Gamma Function)

Beta Function

The Beta function $B(a, b)$ is defined by:

$$B(a,b) = \int_0^1 t^{\,a-1}(1-t)^{\,b-1}\, dt$$

It is related to the Gamma function by:

$$B(a,b) = \frac{\Gamma(a)\,\Gamma(b)}{\Gamma(a+b)}$$

[26]:
```
def beta(a, b):
    return math.gamma(a) * math.gamma(b) / math.gamma(a + b)

for a, b in [(1, 1), (2, 3), (0.5, 0.5)]:
    print(f'B({a}, {b}) = {beta(a, b):.6f}')
```

```
B(1, 1) = 1.000000
B(2, 3) = 0.083333
B(0.5, 0.5) = 3.141593
```

Factorial and Related Functions

For a nonnegative integer n, the notation $n!$ is pronounced *n-factorial* and is defined by $0! = 1$ and for $n > 0$:

$$n! = n(n-1)(n-2)\cdots 2 \cdot 1$$

It is closely related to the Gamma function: $n! = \Gamma(n+1)$. (See: Gamma Function)

The notation $n!!$ is the *double factorial*; the factors descend by two. For example, $7!! = 7\times5\times3\times1 = 105$.

[27]:
```
for n in range(8):
    print(f'{n}! = {math.factorial(n)}')
```

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
```

[28]:
```
# double factorial
def double_factorial(n):
    result = 1
    while n > 0:
        result *= n
        n -= 2
    return result

for n in [1, 3, 5, 7, 8, 10]:
    print(f'{n}!! = {double_factorial(n)}')
```

```
1!! = 1
3!! = 3
5!! = 15
7!! = 105
8!! = 384
10!! = 3840
```

The notations $x^{(k)}$ and $x_{(k)}$ are the *rising factorial* and *falling factorial* respectively:

$$x^{(k)} = x(x+1)\cdots(x+k-1) = \prod_{j=0}^{k-1}(x+j)$$

$$x_{(k)} = x(x-1)\cdots(x-k+1) = \prod_{j=0}^{k-1}(x-j)$$

Note that $x^{(0)} = x_{(0)} = 1$. The rising factorial is also called the *Pochhammer symbol* and is sometimes written $(x)_k$.

[29]:
```
def rising_factorial(x, k):
    result = 1
    for j in range(k):
        result *= (x + j)
    return result

def falling_factorial(x, k):
    result = 1
    for j in range(k):
        result *= (x - j)
    return result

x, k = 3, 4
print(f'x = {x}, k = {k}')
print(f'rising  x^(k) = {rising_factorial(x, k)}')
print(f'falling x_(k) = {falling_factorial(x, k)}')
```

```
x = 3, k = 4
rising  x^(k) = 360
falling x_(k) = 0
```

Binomial Coefficient

The binomial coefficient $\binom{n}{k}$ is defined for nonnegative integers n and k with $0 \le k \le n$ as the number of k-element subsets of an n-element set.

$$\binom{n}{k} = \frac{n!}{k!\,(n-k)!}$$

For $k > n$, $\binom{n}{k} = 0$. The letter C is also used: $C(n,k)$, C_n^k, ${}_nC_k$. In this context C stands for *combinations*: the number of ways to choose k items from n.

[30]:
```
for n in range(6):
    row = [math.comb(n, k) for k in range(n + 1)]
    print(f'n={n}: {row}')
```

```
n=0: [1]
n=1: [1, 1]
n=2: [1, 2, 1]
n=3: [1, 3, 3, 1]
n=4: [1, 4, 6, 4, 1]
n=5: [1, 5, 10, 10, 5, 1]
```

(See: Factorial and Related Functions)

(See: Binomial Coefficient)

Multinomial Coefficient

For nonnegative integers $n, k_1, k_2, \ldots, k_l$ with $k_1 + k_2 + \cdots + k_l = n$, the multinomial coefficient is:

$$\binom{n}{k_1,\, k_2,\, \ldots,\, k_l} = \frac{n!}{k_1!\, k_2! \cdots k_l!}$$

It counts the number of ways to partition n items into l groups of sizes $k_1, k_2, \ldots, k_l$.

[31]:
```
def multinomial(n, ks):
    result = math.factorial(n)
    for k in ks:
        result //= math.factorial(k)
    return result

# ways to arrange the letters of "MISSISSIPPI"
# M:1, I:4, S:4, P:2 => n=11
print(multinomial(11, [1, 4, 4, 2]))
```

```
34650
```

Divisibility

For integers a and $b \neq 0$, the expression $b \mid a$ means "b divides a," i.e., there exists an integer q such that $a = bq$. It is read as "b divides a."

$$b \mid a \iff \exists\, q \in \mathbb{Z},\ a = bq$$

[32]:
```
def divides(b, a):
    return a % b == 0
```

```
for a in [12, 15, 20, 25]:
    print(f'5 | {a}: {divides(5, a)}')
```

```
5 | 12: False
5 | 15: True
5 | 20: True
5 | 25: True
```

(See: Divisibility)

Modular Arithmetic

For integers a, b with $b > 0$, the expression $a \bmod b$ is the remainder when a is divided by b. Specifically, dividing a by b yields integers q (quotient) and r (remainder) such that:

$$a = qb + r, \qquad 0 \leq r < b$$

In addition to the function mod, there is a relation called *congruence* that employs the term mod. For a positive integer n and any integers a and b:

$$a \equiv b \pmod{n}$$

means $a - b$ is divisible by n, i.e., $n \mid (a - b)$.

[33]:
```
# a mod b
print('13 mod 10 =', 13 % 10)
print('-10 mod 9 =', -10 % 9)

# congruence: a  b (mod n)
a, b, n = 17, 5, 6
print(f'{a}  {b} (mod {n}): {(a - b) % n == 0}')
```

```
13 mod 10 = 3
-10 mod 9 = 8
17  5 (mod 6): True
```

GCD and LCM

For positive integers n and m, the notation $\gcd(n, m)$ is the *greatest common divisor* — the largest positive integer that divides both n and m. In number theory this is abbreviated to (n, m).

The notation $\operatorname{lcm}(n, m)$ is the *least common multiple* — the smallest positive integer that is divisible by both n and m.

$$\gcd(n, m) \cdot \operatorname{lcm}(n, m) = n \cdot m$$

[34]:
```
for n, m in [(12, 18), (35, 14), (17, 13)]:
    g = math.gcd(n, m)
    l = math.lcm(n, m)
    print(f'gcd({n}, {m}) = {g},  lcm({n}, {m}) = {l},  product check: {g * l␣
 ↪== n * m}')
```

```
gcd(12, 18) = 6,  lcm(12, 18) = 36,  product check: True
gcd(35, 14) = 7,  lcm(35, 14) = 70,  product check: True
gcd(17, 13) = 1,  lcm(17, 13) = 221,  product check: True
```

Euler's Totient Function

For a positive integer n, Euler's totient function $\varphi(n)$ is the number of integers in $\{1, 2, \dots, n\}$ that are relatively prime to n (i.e., $\gcd(k, n) = 1$). The symbol φ is a stylized Greek phi; some authors use the ordinary ϕ.

$$\varphi(n) = \left|\{k \in \{1, \dots, n\} : \gcd(k, n) = 1\}\right|$$

[35]:
```
def euler_totient(n):
    count = 0
    for k in range(1, n + 1):
        if math.gcd(k, n) == 1:
            count += 1
    return count

for n in [1, 6, 7, 12, 15, 20]:
    print(f'phi({n:2}) = {euler_totient(n)}')
```

```
phi( 1) = 1
phi( 6) = 2
phi( 7) = 6
phi(12) = 4
phi(15) = 8
phi(20) = 8
```

Identity Function

For a set A, the *identity function* on A is denoted $\mathrm{Id}_A : A \to A$ and defined by $\mathrm{Id}_A(a) = a$ for all $a \in A$.

$$\mathrm{Id}_A(a) = a$$

[36]:
```
def Id(a):
    return a

A = [1, 'hello', 3.14, True]
```

```
print([Id(a) for a in A])
```

```
[1, 'hello', 3.14, True]
```

Characteristic Function

Given a set A that is a subset of the real numbers, the *characteristic function* of A is denoted χ_A and defined by:

$$\chi_A(x) = \begin{cases} 1 & \text{if } x \in A \\ 0 & \text{otherwise} \end{cases}$$

Some authors write $\mathbf{1}_A$ for the characteristic function of A.

[37]:
```
A = {1, 2, 3, 4, 5}

def chi_A(x):
    return 1 if x in A else 0

for x in range(8):
    print(f'chi_A({x}) = {chi_A(x)}')
```

```
chi_A(0) = 0
chi_A(1) = 1
chi_A(2) = 1
chi_A(3) = 1
chi_A(4) = 1
chi_A(5) = 1
chi_A(6) = 0
chi_A(7) = 0
```

Classes of Functions

Continuous Functions

The notation C^k denotes the class of functions whose k^{th} derivative is defined and continuous. Thus:

- C^0 — class of continuous functions
- C^1 — class of differentiable functions whose derivatives are continuous
- C^2 — class of functions with continuous second derivatives

And so forth. The class C^∞ contains functions for which derivatives of all orders exist.

$$C^0 \supset C^1 \supset C^2 \supset \cdots$$

For functions defined on an interval $[a, b]$, this may be written $C^k([a, b])$. The notation $C^k(\mathbb{R})$ is equivalent to C^k.

Lp Spaces

The notation L^p, where p is a positive real number, denotes the class of functions for which:

$$\int_{-\infty}^{\infty} |f(x)|^p \, dx < \infty$$

This may be restricted to an interval: $L^p([a,b])$ stands for functions on $[a,b]$ satisfying the above condition.

Closely related is the notation ℓ^p for *sequences* $a_0, a_1, a_2, \ldots$ satisfying:

$$\sum_{k=0}^{\infty} |a_k|^p < \infty$$

[38]:
```
# Demonstrate checking partial-sum convergence of a sequence in l^p
# The sequence a_k = 1/k^2 is in l^1 (its sum converges)
N = 10000
partial_sum = sum(1 / k ** 2 for k in range(1, N + 1))

print(f'partial sum of 1/k^2 for k=1..{N}: {partial_sum:.6f}')
print(f'pi^2/6 (exact limit):               {math.pi**2 / 6:.6f}')
```

```
partial sum of 1/k^2 for k=1..10000: 1.644834
pi^2/6 (exact limit):                1.644934
```

Polynomials, Power Series, and Rational Functions

Polynomials

A *polynomial* $p(x)$ is a function of the form:

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

The values $a_0, a_1, \ldots, a_n$ are called the *coefficients*. Some authors prefer the subscripts on the coefficients to run counter to the exponents:

$$p(x) = a_0 x^n + a_1 x^{n-1} + \cdots + a_{n-1} x + a_n$$

Polynomials may have more than one variable, e.g., $p(x, y) = x^3 y^2 - 3xy + 2x - 3$.

[39]:
```
def poly_eval(coeffs, x):
    # coeffs[i] is the coefficient of x^i
    return sum(a * x ** i for i, a in enumerate(coeffs))

# p(x) = 5x^2 + 10x + 20
```

```
coeffs = [20, 10, 5]
x = 2
print(f'p({x}) = {poly_eval(coeffs, x)}')
```

```
p(2) = 60
```

Degree of a Polynomial

The *degree* of a polynomial $p(x)$ is the largest exponent on x associated with a nonzero coefficient. It is denoted:

$$\deg p(x)$$

If $p(x) = 0$ (the zero polynomial), the degree is either undefined or $-\infty$.

For polynomials in two or more variables, the degree of a term is the sum of the exponents of the variables; the *total degree* is the maximum over all terms. For example, if $p(x, y) = x^2y^2 + xy^2 + y^3$, then $\deg p(x, y) = 4$.

[40]:
```
def poly_degree(coeffs):
    # coeffs[i] is coefficient of x^i; find highest nonzero
    for i in range(len(coeffs) - 1, -1, -1):
        if coeffs[i] != 0:
            return i
    return None  # zero polynomial

print('deg(5x^2 + 10x + 20) =', poly_degree([20, 10, 5]))
print('deg(x^4 + 1)         =', poly_degree([1, 0, 0, 0, 1]))
print('deg(0)               =', poly_degree([0]))
```

```
deg(5x^2 + 10x + 20) = 2
deg(x^4 + 1)         = 4
deg(0)               = None
```

Set of All Polynomials

The set of all polynomials in the variable x with real coefficients is denoted:

$$\mathbb{R}[x]$$

More generally, if R is any ring, $R[x]$ is the set of all polynomials in x with coefficients in R.

Polynomials may have more than one variable. The notation $\mathbb{R}[x, y]$ stands for the set of all polynomials with real coefficients in variables x and y.

Polynomial Divisibility and Modular Arithmetic

Just as with integers, the notions of divisibility and modular arithmetic apply to polynomials:

- $p(x) \mid q(x)$ means there is a polynomial $\alpha(x)$ so that $\alpha(x) \cdot p(x) = q(x)$.
- $p(x) \equiv q(x) \pmod{\alpha(x)}$ means that $\alpha(x)$ divides $p(x) - q(x)$.
- $R[x]/(q(x))$ is the set of polynomials with coefficients from R taken modulo $q(x)$. For example, $\mathbb{Z}[x]/(x^2+1)$ is effectively the Gaussian integers.

[41]:
```
# Polynomial long division: (x^3 - 1) / (x - 1) = x^2 + x + 1
# Verify by evaluating at x = 2
x = 2
dividend = x ** 3 - 1
divisor = x - 1
quotient = x ** 2 + x + 1

print(f'(x^3 - 1) at x={x}: {dividend}')
print(f'(x - 1) at x={x}:   {divisor}')
print(f'(x^2+x+1) at x={x}: {quotient}')
print(f'divisor * quotient:  {divisor * quotient}')
```

```
(x^3 - 1) at x=2: 7
(x - 1) at x=2:   1
(x^2+x+1) at x=2: 7
divisor * quotient:  7
```

Special Polynomial Families

Certain families of polynomials have their own notation:

1. **Chebyshev polynomials**: The degree-n Chebyshev polynomial of the first kind is denoted $T_n(x)$ and the second kind $U_n(x)$.
2. **Hermite polynomials**: The degree-n Hermite polynomial is denoted $H_n(x)$. Two conventions exist (probability vs. physics community).
3. **Laguerre polynomials**: The degree-n Laguerre polynomial is denoted $L_n(x)$.
4. **Legendre polynomials**: The degree-n Legendre polynomial is denoted $P_n(x)$.

[42]:
```
# Chebyshev polynomials of the first kind T_n(x)
# T_0(x)=1, T_1(x)=x, T_n(x) = 2x T_{n-1}(x) - T_{n-2}(x)
def chebyshev_T(n, x):
    if n == 0: return 1
    if n == 1: return x
    t0, t1 = 1, x
    for _ in range(2, n + 1):
        t0, t1 = t1, 2 * x * t1 - t0
    return t1

x = 0.5
for n in range(6):
```

```
    print(f'T_{n}({x}) = {chebyshev_T(n, x):.4f}')
```

```
T_0(0.5) = 1.0000
T_1(0.5) = 0.5000
T_2(0.5) = -0.5000
T_3(0.5) = -1.0000
T_4(0.5) = -0.5000
T_5(0.5) = 0.5000
```

Power Series

A *power series* is a function of the form:

$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots = \sum_{k=0}^{\infty} a_k x^k$$

The set of all power series with coefficients drawn from a ring R is denoted $R[[x]]$. These are sometimes called *formal* power series when convergence is not at issue.

[43]:
```
# e^x as a power series: sum of x^k / k!
def exp_series(x, terms=15):
    return sum(x ** k / math.factorial(k) for k in range(terms))

x = 1.0
print(f'exp_series({x}, 15 terms) = {exp_series(x):.10f}')
print(f'math.exp({x})             = {math.exp(x):.10f}')
```

```
exp_series(1.0, 15 terms) = 2.7182818285
math.exp(1.0)             = 2.7182818285
```

Rational Functions

A *rational function* is the ratio of two polynomials:

$$r(x) = \frac{a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0}{b_m x^m + b_{m-1} x^{m-1} + \cdots + b_1 x + b_0}$$

The set of all rational functions with real coefficients is denoted $\mathbb{R}(x)$. More generally, if F is any field, $F(x)$ is the set of rational functions in x with coefficients drawn from F.

[44]:
```
# r(x) = (x^2 + 1) / (x - 3)
def r(x):
    return (x ** 2 + 1) / (x - 3)

for x in [0, 1, 2, 4, 5]:
    print(f'r({x}) = {r(x):.4f}')
```

```
r(0) = -0.3333
r(1) = -1.0000
r(2) = -5.0000
r(4) = 17.0000
r(5) = 13.0000
```

Miscellany

Arg Min and Arg Max

Given a function f, the notation $\arg\max_x f(x)$ stands for the value of x that makes $f(x)$ as large as possible. Similarly, $\arg\min_x f(x)$ is the value of x that minimizes $f(x)$.

$$\arg\max_x f(x) \qquad \arg\min_x f(x)$$

A more formal definition: given a set S and $f : S \to \mathbb{R}$,

$$\arg\max_{x \in S} f(x) := \{x \in S \mid f(s) \leq f(x)\ \forall\, s \in S\}$$

The $\arg\min$ / $\arg\max$ may not be unique, defined, or a singleton.

For more information on the defined $:=$ symbol, see the Numbers notebook.

For more information on the for-all $\forall$ symbol, see the Logic notebook.

[45]:
```python
def f(x):
    return -(x ** 2) + 10

X = [-3, -2, -1, 0, 1, 2, 3]
values = [(x, f(x)) for x in X]
max_val = max(v for _, v in values)
argmax = [x for x, v in values if v == max_val]

print('X:   ', X)
print('f(X):', [f(x) for x in X])
print(f'argmax f(x) = {argmax}, with f(x) = {max_val}')
```

```
X:    [-3, -2, -1, 0, 1, 2, 3]
f(X): [1, 6, 9, 10, 9, 6, 1]
argmax f(x) = [0], with f(x) = 10
```

One-to-One, Onto, and Bijection

Sometimes the arrow in $f : A \to B$ is modified to indicate special properties:

- **One-to-one (injection):** $f : A \hookrightarrow B$ — distinct inputs always produce distinct outputs. No two elements of A map to the same element of B.

- **Onto (surjection):** $f : A \twoheadrightarrow B$ — every element of B is the output of at least one element of A, i.e., $\text{Im}(f) = B$.
- **Bijection:** A function that is both one-to-one and onto. A bijection establishes a perfect pairing between A and B.

$$\text{One-to-one:} \quad f(x_1) = f(x_2) \implies x_1 = x_2$$

$$\text{Onto:} \quad \forall\, y \in B,\ \exists\, x \in A \text{ such that } f(x) = y$$

[46]:
```
def is_injective(f, domain):
    outputs = [f(x) for x in domain]
    return len(outputs) == len(set(outputs))

def is_surjective(f, domain, codomain):
    image = {f(x) for x in domain}
    return image == codomain

A = {-2, -1, 0, 1, 2}
B = {0, 1, 4}

print('f(x) = x^2')
print('injective:', is_injective(lambda x: x**2, A))
print('surjective:', is_surjective(lambda x: x**2, A, B))
```

```
f(x) = x^2
injective: False
surjective: True
```

[47]:
```
# g(x) = 2x + 1 on integers: injective
A = {0, 1, 2, 3, 4}
B = {1, 3, 5, 7, 9}

print('g(x) = 2x + 1')
print('injective:', is_injective(lambda x: 2*x + 1, A))
print('surjective:', is_surjective(lambda x: 2*x + 1, A, B))
print('bijection:', is_injective(lambda x: 2*x + 1, A) and is_surjective(lambda␣
 ↪x: 2*x + 1, A, B))
```

```
g(x) = 2x + 1
injective: True
surjective: True
bijection: True
```